

SLIIT
SE3082 – Parallel Computing



REPORT
LAB 05

Akash D
IT24101853

Exercise 02

Code Implementation

```
M /a/3 Semester/PC/Lab05/Exercise02
GNU nano 9.1
#include <omp.h>
#include <stdio.h>

int main() {

#pragma omp parallel
{
    int th = omp_get_thread_num();
    printf("thread # %d\n", th);

#pragma omp single
    {
        printf("Inside Single # %d\n", omp_get_thread_num());
        printf("Exiting Single \n");
    }
    printf("Hi again from thread # %d\n", omp_get_thread_num());
}

    return 0;
}
```

Output

```
M /a/3 Semester/PC/Lab05/Exercise02

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise01
$ cd "A:\3 Semester\PC\Lab05"

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05
$ cd Exercise02

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise02
$ nano lab05b.c

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise02
$ gcc -fopenmp lab05b.c -o lab05b.o

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise02
$ ./lab05b.o
thread # 1
Inside Single # 1
thread # 3
thread # 5
thread # 4
thread # 6
thread # 7
thread # 0
thread # 13
thread # 12
thread # 8
thread # 10
thread # 14
thread # 9
thread # 15
thread # 11
thread # 2
Exiting Single
Hi again from thread # 3
Hi again from thread # 6
Hi again from thread # 5
Hi again from thread # 4
Hi again from thread # 7
Hi again from thread # 10
Hi again from thread # 0
Hi again from thread # 13
Hi again from thread # 12
Hi again from thread # 14
Hi again from thread # 8
Hi again from thread # 9
Hi again from thread # 11
Hi again from thread # 2
Hi again from thread # 15
Hi again from thread # 1

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise02
$ |
```

In OpenMP `#pragma omp single` specifies that the enclosed code section is executed by only one single thread in the team rather than all threads. The first thread that arrives at the single construct executes it while other threads skip it and continue to the end of the construct where an implicit barrier exists by default unless a `nowait` clause is specified.

In this specific program:

1. Parallel Execution: When the parallel region starts multiple threads execute concurrently printing their respective thread numbers.
2. Single Construct Execution: Thread 1 reaches the single directive first and executes the print statements inside it showing Inside Single 1 and Exiting Single. Other threads bypass this block.
3. Implicit Barrier: All threads synchronize at the end of the single construct before proceeding to the subsequent statements ensuring orderly execution of the remaining